Joint Task Offloading, Resource Allocation and Model Placement for AI as a Service in 6G Network

Yuhao Chai, Kaice Gao, Guohan Zhang, Lu Lu, *Member, IEEE*, Qin Li, and Yong Zhang

***Abstract*—In the future, 6G network is expected to achieve deep integration of communication and computation, where computation-centric services will be ubiquitous in the network. There are differences in data size, computing power types (CPU/GPU), model complexity, and Quality of Service (QoS) requirements among various CPU computing services and artificial intelligence (AI) services. By providing AI as a Service (AIaaS) in 6G network, the deployment of AI models and the scheduling of task and computing resources can be accelerated. The fundamental challenge lies in the effective amalgamation of the long-term strategy of the model placement problem and the short-term strategy of the task scheduling problem to attain dynamic scheduling and management of tasks and heterogeneous computing resources. A two-timescale optimization method for joint task offloading, computing resource allocation and model placement is proposed in this article. We present an edge-network-cloud framework that configures AIaaS functional units, taking into account the heterogeneous computing requirements and QoS demands of different services. A long-term problem to minimize latency and energy consumption is formulated. To work out the coupled optimization parameters, the problem is decomposed into short-term deterministic sub-problems using Lyapunov optimization. We propose low-complexity algorithms for joint task offloading strategy based on deferred acceptance algorithm, computing resource allocation strategy based on convex optimization, and model placement strategy based on multi-armed bandits. Experimental results demonstrate that our approach outperforms reinforcement learning and other popular optimization algorithms in terms of complexity and effectiveness.**

***Index Terms*—6G network, resource allocation, AI as a service (AIaaS), Lyapunov optimization, deferred acceptance algorithm, multi-armed bandit.**

Received 19 January 2024; revised 10 August 2024; accepted 14 August 2024. Date of publication 28 August 2024; date of current version 30 December 2024. This work was supported by the Beijing University of Posts and Telecommunications-China Mobile Research Institute Joint Innovation Center. *(Corresponding author: Yong Zhang.)*

Yuhao Chai and Kaice Gao are with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: chaiyh@bupt.edu.cn; gaokaice@bupt.edu.cn).

Guohan Zhang is with the School of Science and Key Laboratory of Mathematics and Information Networks, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: zhanggh@bupt.edu.cn).

Lu Lu and Qin Li are with the Department of Basic Network Technology Research, China Mobile Research Institute, Beijing 100876, China (e-mail: lulu@chinamobile.com; liqinyjy@chinamobile.com).

Yong Zhang is with the School of Electronic Engineering and Beijing Key Laboratory of Work Safety Intelligent Monitoring, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: yongzhang@bupt.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TSC.2024.3451170, provided by the authors.

Digital Object Identifier 10.1109/TSC.2024.3451170

I. Introduction

NETWORK intelligence is one of the visions for future 6G networks. Through the deep integration of communication and computation, 6G networks will support massive computing and artificial intelligence (AI) services, providing enhanced communication and connectivity capabilities. The computing resource of 6G networks will extend from "edge-cloud" to "edge-network-cloud" framework, using ubiquitous computing resource in the network to offer stable and efficient AI services to users. Therefore, 6G networks will flexibly allocate computing resources based on connection, computing power, model and data requirements, thus improving network energy efficiency.

By utilizing distributed heterogeneous computing resources, 6G has the potential to provide low-latency services, alleviate communication and computing burdens on clouds, and meet the Quality of Service (QoS) requirements of various communication and computing services [1]. 6G aims to connect diverse communication, computation, data, and AI model resources between the edge, network and cloud, enabling on-demand resource scheduling. International Telecommunication Union (ITU) proposed that the 6G network needs to handle intelligent services such as International Mobile Telecommunications-2030 (IMT-2030) assisted automated driving, AI assisted extreme Multiple-Input Multiple-Output (E-MIMO), offloading of heavy computation operations across devices and networks, creation of and prediction with digital twins, and others [2]. Combined with the concept of edge intelligence, the evolution of telecom infrastructures towards 6G will consider highly distributed AI, moving the intelligence from the central cloud to edge computing resources [3], [4]. This requires 6G network to efficiently manage dedicated computing resources and models for different types of services, enabling unified and efficient resource management.

Providing AI as a Service (AIaaS) in 6G network enables users to get latency-guaranteed AI services without developing their own AI systems [5]. In order to meet the varying Quality of AI Service (QoAIS) requirements of users, new evaluation system needs to be considered on the basis of existing QoS system. Therefore, the provision of AIaaS in 6G network needs comprehensively consider the following aspects of the services:

Latency: Latency is a critical factor that influences service quality for both computing tasks and AI tasks. Latency typically consists of three components: communication latency, queuing latency, and computing latency. Communication latency is

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

influenced by the amount of data and the communication quality. Queuing latency is affected by the length of task queues in computing nodes. Computing latency is influenced by factors such as task workload, model parameters, and remaining computing power in the nodes. Designing appropriate task scheduling algorithms can greatly improve the users' experience for latency-sensitive services.

Energy Consumption: Data transmission and computation in AI tasks result in significant energy consumption. Due to the limited computing power of each server, a trade-off must be considered between energy consumption and latency to ensure both improved energy efficiency and meeting the users' latency requirements [6]. Task offloading decisions and computing resource allocation are the primary research directions for optimizing energy consumption.

Heterogeneous Computing Demands: Different types of tasks require different types and scales of computing resources, such as CPUs, GPUs, TPUs, etc., to support their computing needs. Large-scale training and inference tasks often require GPU clusters rather than a single GPU for computation. CPU-GPU heterogeneous computing is also employed to enhance the computing and latency performance of AI models [7]. Consequently, the coupling of heterogeneous computing resource demands complicates the allocation problem. Computing nodes need serve multiple tasks simultaneously and deploy multiple containers to reduce communication costs or balance workload, presenting new challenges for dynamic resource management and task offloading [8]. Thus intelligent orchestration solutions is helpful to support the constantly growing requirements of intelligent applications [9], [10].

Model Requirements: Artificial intelligence tasks usually necessitate the delegation of duties to servers that possess ample computing resources to carry out the tasks through particular models [11]. With the emergence of techniques such as pruning and early-exit, multiple variants of models may complete the same AI task, with varying parameter sizes and architectures that affect inference accuracy and computing time. Currently, most existing AI services lack a balance between efficiency and performance during deployment [12]. Supporting AI tasks in 6G network requires considering the deployment of multiple models, so as to provide customized QoS for users. On one hand, it is necessary to ensure that tasks with different QoS requirements are offloaded to nearby nodes with appropriate models to guarantee overall latency. On the other hand, it is also essential to ensure that nodes have sufficient computing hardware and facilities to support the deployment of models with different complexities.

According to the different types of computing power used, we can divide 6G services into CPU based computing services and GPU based AI services, hereinafter referred to as computing tasks and AI tasks. In summary, to realize 6G network that supports both existing computing tasks and AI tasks, dynamic task offloading, computing resource allocation, and model placement strategies must be designed, as shown in Fig. 1, short-time strategies and long-time strategies mutual influence on each other's subsequent decisions. AI models are deployed by transferring containerized instances from cloud nodes or model repositories to computing nodes that are equipped with CPUs or GPUs. Tasks are offloaded from edge nodes to computing node, with certain tasks necessitating specific models and computing resource for accurate calculations. The task offloading (including node selection and model selection) and computing resource allocation need to be considered in every time slot as a short timescale problem, while model placement is decided with several time slots interval, as a long timescale problem. In this paper, we focus on the AIaaS architecture for the aforementioned multi-service 6G network, which comprehensively ensures the service quality of computing and AI tasks from various dimensions such as communication, computation, data, and AI model resources [13]. The above strategies can be formulated as a two-timescale mixed-integer nonlinear programming (MINLP) problem, aiming to minimize latency and energy consumption while meeting users' QoS requirements. Specifically, the contributions of this paper are as follows:

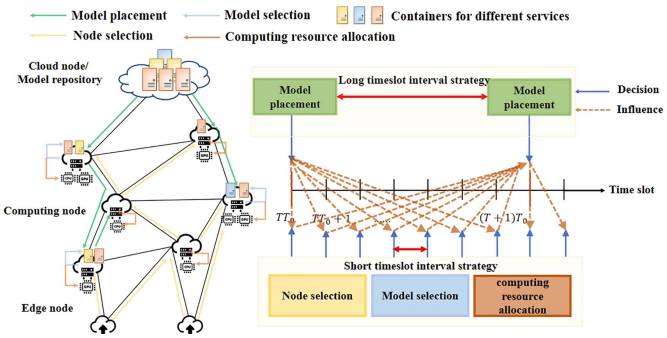


Fig. 1. Strategy decision at different timescales.

1) The functional units of AIaaS in 6G network is constructed, establishing the communication, queuing, and computing models for joint task offloading, computing resource allocation, and model placement in the two-timescale within the edge-network-cloud architecture. To address the heterogeneity of computing resources in the 6G network, we consider different types and scales of computing power in computing nodes, as well as different models and QoAIS features.

2) The Lyapunov optimization is employed to decompose the long-term system stability problem into short-term deterministic sub-problems for minimizing latency and energy consumption. A task offloading mechanism based on deferred acceptance (DA) algorithm is proposed, achieving optimal assignment of different types of services among computing nodes. To ensure differentiated QoS requirements, a dynamic workload allocation strategy and a frequency strategy among heterogeneous computing units on computing nodes based on convex optimization theory are proposed.

3) To address the diverse QoAIS and model selection requirements for AI tasks, a joint model placement and model selection strategy is proposed based on a long-short-term mechanism. In the absence of prior knowledge of future task information, the multi-armed bandit (MAB) mechanism is used to periodically evaluate the model placement strategy on computing nodes, improving the system's communication and computation efficiency by re-deploying models.

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

The rest of the paper is summarized as follows. In Section II, we introduce related work. Section III is system model and problem formulation. Section IV is the process of problem solving. Section V is the simulation results to evaluate our deferred acceptance-multi-armed bandits (DA-MAB) strategy. Finally, we summarize our work in Section VI.

II. Related Work

A. Task Offloading and Resource Allocation

Task offloading and resource allocation have been extensively studied in scenarios such as edge computing, fog computing, and cloud computing. Edge intelligence is an emerging computing and communication paradigm that enables AI functionality at the network edge [14]. To alleviate the computing burden on the end and cloud sides, computing tasks are offloaded to edge nodes, cloud nodes, or data centers for computation [15]. Communication and computing resources, such as transmit power, channel and computing resource, are optimized based on different scenarios. Convex optimization, game theory, and Alternating Direction Method of Multipliers (ADMM) methods have been applied to optimize continuous resource allocation problems. Matching theory, integer optimization, particle swarm optimization (PSO), and other methods have been applied to optimize discrete task offloading problems. Besides, reinforcement learning (RL), including MAB theory, deep Q-network (DQN), Proximal Policy Optimization (PPO), multi-agent learning, etc., is playing a growing and important role in resource allocation problems for the edge [16].

In the field of fine-grained computing power management and accelerated computation of AI models, multi-GPU and CPU-GPU cooperation have emerged as new research trends. A clustered multi-core platform was proposed to handle the energy consumption model during intra-task processing [17]. Zeng et al. deployed edge learning in an architecture that utilizes parallel computing using both CPU and GPU, dynamically allocate bandwidth, divide CPU-GPU workloads, and perform speed scaling at each device [18]. A similar problem was addressed in [19], where the Lagrangian dual method was employed to obtain optimal processor frequency and dataset size allocations. The fine-grained task deployment problem in multi-CPU-GPU cores was tackled using a cooperative dual-population evolutionary algorithm, maximizing average QoS [20]. Fine-grained resource management and heterogeneous collaborative computing contribute to further reducing task execution time and energy consumption.

B. Model Placement

Due to the heavy communication and energy costs associated with frequent model re-deployment from cloud nodes or model repositories, the model placement problem is typically a long timescale problem compared to task offloading. The model placement problem shares commonalities between model training optimization and model inference optimization. The optimization objective is typically to reduce overall latency and energy consumption and is sometimes combined with distributed computing techniques. Even if there are complex dependencies between these training or inference tasks, they can still be modeled as directed acyclic graphs and transformed into chain-like or independent temporal tasks [21].

On the edge side, model placement is often jointly optimized with model partitioning. For example, MAB is used to intelligently determine the semantic and layer-wise splitting strategies of edge-end models [22]. The concept of edge AIaaS is proposed to improve the energy and latency performance of AI services through management of data quality and model complexity [5]. Additionally, placement of the edge servers is also helpful for the time and energy optimization in edge computing [23].

C. Two-Timescale Resource Allocation

The two-timescale optimization problem involves multiple network management in two time dimensions. It often requires effective problem decomposition and the design of different algorithms to optimize each sub-problem. Wei et al. [24] proposed a two-timescale optimization algorithm for resource placement and task dispatching in edge-clouds. Similar problems in related scenarios were addressed in [25], where a low-complexity optimization scheme combining Lyapunov optimization, ADMM, and matching theory was used. In Fog Radio Access Networks, small-scale computing resource allocation problems were optimized using coalition game theory, while large-scale service caching problems were addressed using multi-agent RL [26]. The long-term stochastic optimization problem in non-orthogonal multiple access edge computing was decomposed into three sub-problems: resource block allocation on a long timescale, task partitioning and computing resource allocation on a short timescale. These sub-problems were solved using swap matching and relaxed convex optimization. The long-term collaborative inference problem in cloud-edge-end was transformed into a deterministic problem for each time slot using Lyapunov optimization. By combining deep reinforcement learning (DRL) and low-complexity algorithms, the training complexity of the DRL was reduced [27].

III. System Model and Problem Formulation

A. System Model

In this study, a three-tier edge-network-cloud 6G network architecture is considered to support both computing and AI services in an AIaaS framework. As shown in Fig. 2, tasks are aggregated at edge nodes and transmitted to computing nodes or cloud nodes for processing. Different types of services have distinct QoS requirements. For instance, communication protocol processing and computing analysis heavily rely on the CPU computing capabilities. Training tasks demand more GPU computing power, while inference tasks require low latency and inference accuracy. To enhance the efficiency of AI services, we assume that some computing nodes support GPU-CPU acceleration for training and inference tasks.

To achieve unified orchestration and management of tasks, it is important to first segment the user's tasks and monitor the

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

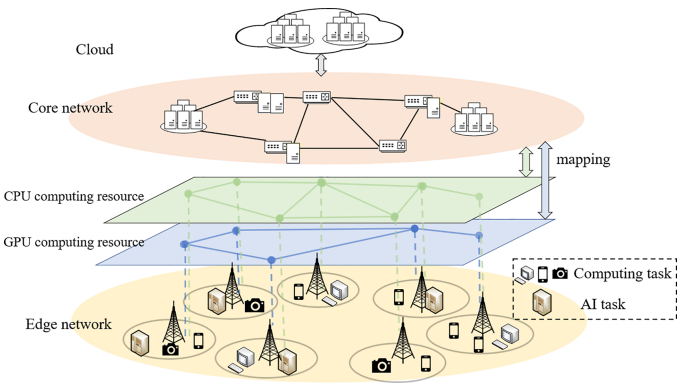


Fig. 2. System architecture of the network.

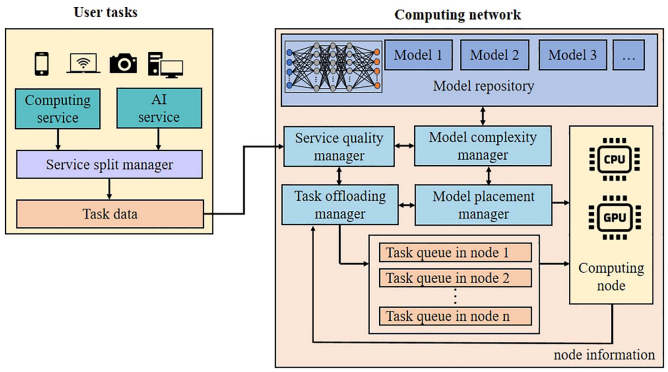


Fig. 3. AIaaS framework in 6G network.

QoS. The overall user task may consist of multiple sub-tasks, and by dividing the task into independent sub-tasks with different QoS and computing power requirements, it facilitates the development of fine-grained orchestration strategies. By setting up managers in the network to collect information about the network and nodes, and dynamically replace models and offload tasks, users offload computing services on nearby nodes using appropriate models while ensuring QoS requirements, which helps prevent wastage of computing resources and overload situations on nodes.

To accommodate the diverse requirements of these services, configurable functional units are introduced, as shown in Fig. 3. The notations and symbols are given in Table I. These units include:

Service Split Manager: The service split manager is responsible for dividing tasks with dependencies or distributed characteristics into a series of single-destination node task sequences with temporal relationships. We divide the task according to the segmentation strategy in [21]. Additionally, the service split manager is responsible for categorizing the split tasks. All tasks can be divided into a set $\boldsymbol{\mathcal{I}} = \{1, 2, \ldots, I\}$, where task i arrives in time slots $\boldsymbol{T} = \{0, 1, 2, \ldots, T\}$, the duration of each time slot is T_{slot}, $C_i(t)$ represents the data of task i at time slot t, $\forall i \in \boldsymbol{\mathcal{I}}$.

Service Quality Manager: The service quality manager is responsible for receiving information from service split manager and managing the QoS requirements of services. Specifically, $\gamma_i^C(t)$ and $\gamma_i^G(t)$ determine the number of CPU and GPU cores needed to execute task i. The maximum allowable delay for task i is $D_i(t)$. Additionally, q_i^{min} is an optional parameter for AI inference services, representing the accuracy requirement of task i.

TABLE I
NOTATIONS AND SYMBOLS

$\boldsymbol{\mathcal{I}}$	The set of tasks, $\boldsymbol{\mathcal{I}} = \{1, 2, \ldots, I\}$;
$\boldsymbol{\mathcal{J}}$	The set of computing nodes, $\boldsymbol{\mathcal{J}} = \{1, 2, \ldots, J\}$;
$\boldsymbol{\mathcal{K}}$	The set of machine learning (ML) models, $\boldsymbol{\mathcal{K}} = \{0, 1, 2, \ldots, K\}$. Specifically, 0 represents need no model for computing;
$\boldsymbol{\mathcal{K}}_j$	The set of ML models that can be deployed at node j, $\forall j \in \boldsymbol{\mathcal{J}}$, $\boldsymbol{\mathcal{K}}_j \subset \boldsymbol{\mathcal{K}}$;
$C_i(t)$	The data size of task i at time t;
$D_i(t)$	The delay requirement of task i at time t;
$\gamma_i^C(t)$	CPU core requirement of task i at time t;
$\gamma_i^G(t)$	GPU core requirement of task i at time t;
q_i^{min}	Lower bound of accuracy requirement of task i;
α_j^{max}	Upper limit of deployment model amount for node j;
$Q_j^{C,max}$	Total CPU cores in node j;
$Q_j^{G,max}$	Total GPU cores in node j;
$R_{i,j}(t)$	The link transmission rate from task i to node j at time t;
$d_{i,j}$	The number of hops from task i to node j;
$Z_{i,j}(t)$	The queue length of task i at node j at time t;
L	The processing density of standard computing tasks;
a_k	The inference accuracy of model k;
b_k	The ratio of the processing density of model k to the processing density L of standard computing tasks;
$v_{i,k}$	The binary variable determines whether task i can use model k to calculate;
$\alpha_{j,k}(t)$	The binary variable determines whether model k is deployed at node j at time t;
$\beta_{i,j,k}(t)$	The binary variable determines whether task i use model k at node j to compute at time t;
$W_{i,j}^C(t)$	The workload allocated to the CPU by task i at node j at time t;
$W_{i,j}^G(t)$	The workload allocated to the GPU by task i at node j at time t;
$f_{i,j}^C(t)$	The CPU computing frequency of task i running at node j at time t;
$f_j^{C,max}$	The upper limit of CPU computing frequency at node j;
$f_{i,j}^G(t)$	The GPU computing frequency of task i running at node j at time t;
$f_j^{G,max}$	The upper limit of GPU computing frequency at node j;
ε_j^C	The CPU calculation energy consumption coefficient of node j;
ε_j^G	The GPU calculation energy consumption coefficient of node j;
ε_i^d	The transmission energy consumption coefficient of task i;

Model Complexity Manager: The model complexity manager is responsible for managing the model repository and parameters of AI models. The model repository includes models applicable to different types of AI services, as well as different versions of the same model. The set of AI models is denoted as $\boldsymbol{\mathcal{K}} = \{0, 1, 2, \ldots, K\}$, where $k = 0$ represents no model is needed for task computing. The QoAIS for AI model k include accuracy a_k and model complexity ratio b_k. Accuracy measures the inference quality of AI inference services, while the model complexity ratio is the ratio of the processing density of model k to the processing density L (in cycles/bit) of standard computing tasks. The remaining parameters related to QoS will be introduced in the following managers.

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

B. Decision Manager

Decision manager consists of the task offloading manager and the model placement manager, responsible for periodically updating model placements on each computing node and devising node selection and resource allocation strategies for tasks in each time slot.

Task Offloading Manager: The task offloading manager regularly interacts with the service quality manager to obtain QoS requirements and communicates with computing nodes to acquire node computing capabilities and communication conditions. The task offloading manager receives node information related to a set of computing nodes $\mathcal{J} = \{1, 2, \ldots, J\}$, where node j has CPU core count $Q_j^{C,\max}$, maximum CPU operating frequency $f_j^{C,\max}$, GPU core count $Q_j^{G,\max}$, maximum GPU operating frequency $f_j^{G,\max}$, maximum number of deployed models $\alpha_j^{\max}$, set of deployable models $\mathcal{K}_j$ at node j, the current set of models $\{\alpha_{j,k}(t)\}$ deployed on node j, task queue set $\{Z_{i,j}(t)\}$ for task i at node j, CPU computing energy coefficient ε_j^C at node j, and GPU computing energy coefficient ε_j^G at node j. Here, $\alpha_j^{\max}$ and $\mathcal{K}_j$ are determined by the computing, memory and storage capacity of node j, where nodes with larger computing capacity can deploy a greater variety and number of candidate models.[1] Communication information includes network topology, the set of link transmission rate $\{R_{i,j}(t)\}$, and the set of hops $\{d_{i,j}\}$.

At the beginning of each time slot, the task offloading manager determines the task offloading strategy and resource allocation strategy. The decision variables are as follows: At time t, $\beta_{i,j,k}(t) = 1$ denotes that task i uses model k on node j for computation. Specifically, $\beta_{i,j}(t) = \sum_k \beta_{i,j,k}(t) = 1$ indicates that task i is offloaded to node j. The decision variables also include the amount of task workload allocated to the CPU on node j, $W_{i,j}^C(t)$; the amount of task workload allocated to the GPU on node j, $W_{i,j}^G(t)$; the optimal operating frequency of the CPU for task i on node j, $f_{i,j}^C(t)$; and the optimal operating frequency of the GPU, $f_{i,j}^G(t)$.

Model Placement Manager: The model placement manager regularly communicates with computing nodes and the model complexity manager to coordinate model placement strategies. The model placement manager obtains the indicator $v_{i,k}$ from the model complexity manager. If $v_{i,k} = 1$, it indicates that task i can be computed by model k. The models deployed on nodes can be reused for different tasks in the form of containers [28]. Based on historical information received from the task offloading manager, at the beginning of each T_0 time slots, the model placement manager determines the set of deployed models $\{\alpha_{j,k}(t)\}$ on each node j, where $\alpha_{j,k}(t) = 1$ indicates that model k is deployed on node j at time t and remains unchanged for the next T_0 time slots.

[1]If the deployed models have significantly different parameter sizes, the domain of $\alpha_{j,k}(t)$ can be extended to non-negative real numbers in constraint (1), or set other parameters and constraints to better characterize the hardware requirements of the models. The supposed DA and MAB strategy introduced below supports the above extension.

For the decision managers, the constraints can be expressed as follows:

$$\sum_k \alpha_{j,k}(t) \le \alpha_j^{\max}, \forall j \in \mathcal{J}, \tag{1}$$

Equation (1) indicates that the number of AI model deployed by computing node j does not exceed $\alpha_j^{\max}$. Based on the requirements of the task and limitations of the node computing power, we have constraints

$$\sum_i \beta_{i,j}(t)\gamma_i^C(t) \le Q_j^{C,\max}, \forall j \in \mathcal{J}, \tag{2}$$

$$\sum_i \beta_{i,j}(t)\gamma_i^G(t) \le Q_j^{G,\max}, \forall j \in \mathcal{J}, \tag{3}$$

Equation (2) and (3) indicate that the number of CPU and GPU cores used by all tasks at computing node j at the same time does not exceed the node resource limit. Each task can only be offloaded to one computing node for computation within the same time slot. Therefore, we have constraints

$$\sum_j \beta_{i,j}(t) \le 1, \forall i \in \mathcal{I}. \tag{4}$$

The workload segmentation of each task on the CPU and GPU cores should meet the conditions (5)–(7), where $b_{i,k} = v_{i,k} * b_k$

$$W_{i,j}^C(t) + W_{i,j}^G(t) = \sum_k \beta_{i,j,k}(t) b_{i,k} LC_i(t), \forall i \in \mathcal{I}, \forall j \in \mathcal{J}, \tag{5}$$

$$0 \le W_{i,j}^C(t), \forall i \in \mathcal{I}, \forall j \in \mathcal{J}, \tag{6}$$

$$0 \le W_{i,j}^G(t), \forall i \in \mathcal{I}, \forall j \in \mathcal{J}, \tag{7}$$

where $b_{i,k}LC_i(t)$ is the workload of task i using model k, and the unit of L is cycle/bit. (5) indicates the allocation relationship between the workload of task i on the CPU and GPU, while (6) and (7) specify that the amount of workload is non-negative.

According to the different CPU/GPU types of computing nodes, the optimal operating frequency should not exceed its frequency upper limit

$$f_{i,j}^C(t) \le f_j^{C,\max}, \forall i \in \mathcal{I}, \forall j \in \mathcal{J}, \tag{8}$$

$$f_{i,j}^G(t) \le f_j^{G,\max}, \forall i \in \mathcal{I}, \forall j \in \mathcal{J}. \tag{9}$$

AI inference services should meet condition (10) to meet the minimum inference accuracy requirements of the services. q_i^{min} for other services can be set to 0.

$$\sum_j \sum_k \beta_{i,j,k}(t) a_k \ge q_i^{min}, \forall i \in \mathcal{I}. \tag{10}$$

C. Delay and Energy Consumption Model

After task offloading manager makes the task offloading decision, all tasks are transmitted to the target computing nodes following the shortest path. The total latency of a task consists of transmission latency, queuing latency and computation latency. The total energy consumption of a task consists of transmission energy consumption and computation energy consumption. At

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

time t, the transmission latency for task i can be calculated as follows:

$$D_i^T(t) = \sum_j \beta_{i,j}(t) \frac{d_{i,j} C_i(t)}{R_{i,j}(t)}. \tag{11}$$

The transmission energy consumption is

$$E_i^T(t) = \sum_j \beta_{i,j}(t) \frac{\varepsilon_i^d d_{i,j} C_i(t)}{R_{i,j}(t)}, \tag{12}$$

where ε_i^d is the transmission energy consumption coefficient.

The computation latency is determined by the later completion time between CPU and GPU workload

$$D_i^{Com}(t) = \sum_j \max\left\{\frac{W_{i,j}^C(t)}{\gamma_i^C(t) f_{i,j}^C(t)}, \frac{W_{i,j}^G(t)}{\gamma_i^G(t) f_{i,j}^G(t)}\right\}. \tag{13}$$

The computation energy consumption is

$$E_i^{Com}(t) = \sum_j \left(\varepsilon_j^C W_{i,j}^C(t) \left(f_{i,j}^C(t)\right)^2 + \varepsilon_j^G W_{i,j}^G(t) \left(f_{i,j}^G(t)\right)^2\right), \tag{14}$$

where ε_j^C is the computing energy consumption coefficient for the CPU of node j, ε_j^G is the computing energy consumption coefficient for the GPU of node j.

D. Data Queue Model

The data offloaded by task i to a computing node j can be viewed as a data queue over time. If the amount of data from the task exceeds the computing capacity of the node, it will result in queue buildup and increase the queuing latency for tasks. The task queue for task i at computing node j at time t is $Z_{i,j}(t)$. Based on the data amount of the tasks and the resource allocation strategy, the queues update as

$$Z_{i,j}(t+1) = \max\{Z_{i,j}(t) - x_{i,j}(t), 0\} + y_{i,j}(t), \tag{15}$$

$$x_{i,j}(t) = \sum_k v_{i,k} \beta_{i,j,k}(t) \alpha_{j,k}(t) T_{slot} \cdot \left[\gamma_i^C(t) f_{i,j}^C(t) + \gamma_i^G(t) f_{i,j}^G(t)\right], \tag{16}$$

$$y_{i,j}(t) = \sum_k \beta_{i,j,k}(t) b_{i,k} L C_i(t), \tag{17}$$

where $x_{i,j}(t)$ is the maximum amount of data computing of task i at computing node j during $[t, t+1)$. $y_{i,j}(t)$ is task i required computing power for using model k calculating at node j at time slot $t+1$. The long-term stability conditions of $Z_{i,j}(t)$ are

$$\lim_{t\to\infty} \frac{1}{t} \sum_{\tau=0}^{t-1} E\left[Z_{i,j}(\tau)\right] = 0. \tag{18}$$

According to Little's Law, the average latency is a function of queue length and arrival rate. We define the delay violation probability as the probability that the average queue delay exceeds the delay requirement D_i^Q, and we aim to impose constraints on the delay violation probability. Thus, we decompose the delay requirement $D_i(t)$ into three components: the queue delay upper limit D_i^Q, the time-varying transmission delay $D_i^T(t)$, and the computing delay $D_i^{Com}(t)$. We use D_i^Q as the maximum value of the queue delay and optimize the transmission delay $D_i^T(t)$ and computing delay $D_i^{Com}(t)$ through resource allocation. Consequently, the delay requirement can be formulated as

$$D_i^{Q*}(t) = \frac{\lim_{t\to\infty} \frac{1}{t} \sum_{\tau=0}^{t-1} y_{i,j}(\tau)}{\left\{\lim_{t\to\infty} \frac{1}{t} \sum_{\tau=0}^{t-1} x_{i,j}(\tau)\right\} \left\{\lim_{t\to\infty} \frac{1}{t} \sum_{\tau=0}^{t-1} (x_{i,j}(\tau) - y_{i,j}(\tau))\right\}} \le D_i^Q, \tag{19}$$

$$D_i^T(t) + D_i^{Com}(t) + D_i^Q \le D_i(t). \tag{20}$$

E. Problem Formulation

To maximize the efficiency of the network, we need to perform multivariate optimization at two different time scales: long-term model placement strategy, short-term task offloading and computing resource allocation strategy. The overall objective of the optimization is to minimize latency and energy consumption while ensuring computing resource constraints. Let $f_1(t) = \frac{\mu}{T^*} \sum_i (D_i^T(t) + D_i^{Com}(t)) + \frac{(1-\mu)}{E^*} \sum_i (E_i^T(t) + E_i^{Com}(t))$, then the optimization objective is

$$\begin{aligned} P1: \quad & \min \quad F_1 = \lim_{T\to\infty} \frac{1}{TT_0} \sum_{t=0}^{TT_0} f_1(t) \\ & \text{s.t.} \quad (1)\text{–}(10) \text{ and } (19)\text{–}(20), \end{aligned} \tag{21}$$

where T^* and E^* are normalization coefficients for delay and energy consumption, μ is a adaptive parameters, T_0 is the time interval for executing model placement. There are some challenges in solving problem P1. First, the prior knowledge of data arrivals is unknown. Second, it involves resource allocation at different timescales, as shown in Fig. 1, where node selection, model selection (combined as task offloading), and computing resource allocation need to be jointly optimized at each task arrival interval, while model placement needs to be optimized at each long time slot T_0. Third, P1 is a MINLP that is NP-hard due to the coupling between long-term constraints and short-term optimization objectives. Hence, it is more complex than traditional MINLP.

IV. Problem Solving

In this section, the problem is decomposed by using Lyapunov optimization first. Then obtain the optimal analytical solutions for workload and frequency by using convex optimization and solve the node and model selection problem (task offloading problem) using the improved DA algorithm. Subsequently, a model placement strategy based on the Multi-Armed Bandit-Upper Confidence Bounds (MAB-UCB) algorithm is proposed and integrate the three algorithms to form the overall DA-MAB strategy.

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

A. Problem Transformation Based on Lyapunov Optimization

To solve the long-term problem mentioned above, Lyapunov optimization is introduced to transform the long-term optimization problem into a series of short-term optimization sub-problems, which are further decomposed at two-timescale. Let $Z(t) = \{Z_{1,1}(t), Z_{1,2}(t), \ldots, Z_{I,J}(t)\}$, the Lyapunov function can be written as

$$Ly(t) = \frac{V_1}{2}\left\{\sum_i \sum_j Z_{i,j}^2(t)\right\}, \tag{22}$$

where V_1 is the control parameter of the virtual queue. Lyapunov drift depends on the state of the data queue, as given by the following equation:

$$\Delta_T\left(Z(t)\right) = E\left\{Ly\left(t+T\right) - Ly(t)|Z(t)\right\}. \tag{23}$$

Based on inequality $([a-b]^+ + c)^2 \leq a^2 + b^2 + c^2 + 2a(c-b)$, for each $Z_{i,j}(t)$, we have

$$\begin{aligned} Ly\left(t+1\right) - Ly(t) &= \frac{V_1}{2}\sum_i \sum_j \left[Z_{i,j}^2\left(t+1\right) - Z_{i,j}^2(t)\right] \\ &= \frac{V_1}{2}\sum_i \sum_j \left\{\left(\left[Z_{i,j}(t) - x_{i,j}(t)\right]^+ + y_{i,j}(t)\right)^2 - Z_{i,j}^2(t)\right\} \\ &\leq \frac{V_1}{2}\sum_i \sum_j \left\{x_{i,j}^2(t) + y_{i,j}^2(t) + 2Z_{i,j}(t)\left[y_{i,j}(t) - x_{i,j}(t)\right]\right\}. \end{aligned} \tag{24}$$

Let the result on the right side of the above inequality as $Ly_{new}(t)$. By using the law of telescoping sums in $t \in [TT_0, (T+1)T_0)$ to get

$$Ly\left((T+1)T_0\right) - Ly\left(TT_0\right) \leq \sum_{t=TT_0}^{(T+1)T_0-1} Ly_{new}(t). \tag{25}$$

Therefore, we get

$$\begin{aligned} &\Delta_{T_0}\left(Z(t)\right) = \mathbb{E}\left\{Ly\left(t+T_0\right) - Ly(t)|Z(t)\right\} \\ &\leq \mathbb{E}\left\{\sum_{t=TT_0}^{(T+1)T_0-1} Ly_{new}(t)\middle| Z(t)\right\} \leq \frac{V_1}{2}BT_0 \\ &+ V_1\mathbb{E}\left\{\sum_{t=TT_0}^{(T+1)T_0-1}\sum_i\sum_j Z_{i,j}(t)\left[y_{i,j}(t) - x_{i,j}(t)\right]\middle| Z(t)\right\}, \end{aligned} \tag{26}$$

where $B = I \cdot J \cdot (x_{\max}^2 + y_{\max}^2)$ is a constant. Let $x_{i,j}^C(t) = (\sum_k v_{i,k}\alpha_{j,k}(TT_0)\beta_{i,j,k}(t))\gamma_i^C(t)f_{i,j}^C(t)T_{slot}$, $x_{i,j}^G(t) = (\sum_k v_{i,k}\alpha_{j,k}(TT_0)\beta_{i,j,k}(t))\gamma_i^G(t)f_{i,j}^G(t)T_{slot}$, by using Lyapunov Drift-Plus-Penalty (DPP) method [29], the objective function F_1 in problem P1 is considered a penalty function. Then, the DPP function is represented as

$$\begin{aligned} &\mathbb{E}\left\{\Delta_{T_0}\left(Z(t)\right) + \sum_{t=TT_0}^{(T+1)T_0-1} V_2 f_1(t)\middle| Z(t)\right\} \leq \frac{V_1}{2}BT_0 \\ &+ V_1\mathbb{E}\left\{\sum_{t=TT_0}^{(T+1)T_0-1}\sum_i\sum_j Z_{i,j}(t)\left(y_{i,j}(t) - x_{i,j}(t)\right)\middle| Z(t)\right\} \\ &+ V_2\mathbb{E}\left\{\sum_{t=TT_0}^{(T+1)T_0-1} f_1(t)\middle| Z(t)\right\}. \end{aligned} \tag{27}$$

In the above equation, B is a constant. According to Lyapunov optimization, problem P1 can be transformed into minimize the problem on the right side of the equation in each T_0 time slot and include constraints C1-C10. Therefore, the long-term optimization problem is transformed into a short-term optimization problem as P2.

$$\begin{aligned} P2: \quad &\min\ F_2\left(\alpha_{j,k}, \beta_{i,j,k}, W_{i,j}^C, W_{i,j}^G, f_{i,j}^C, f_{i,j}^G\right) \\ &= \sum_{t=TT_0}^{(T+1)T_0-1}\sum_i\sum_j \left(X_{i,j}(t) + Y_{i,j}(t) + G_{i,j}(t) \right. \\ &\quad \left. + H_{i,j}(t)\right) \\ &\text{s.t.} \quad (1)\text{–}(10) \text{ and } (20) \end{aligned} \tag{28}$$

$$\begin{cases} X_{i,j}(t) = V_1 Z_{i,j}(t) y_{i,j}(t) + V_2\frac{\mu}{T^*}D_i^T(t) + V_2\frac{(1-\mu)}{E^*}E_i^T(t), \\ Y_{i,j}(t) = V_2\frac{\mu}{T^*}D_i^{Com}(t) + V_2\frac{(1-\mu)}{E^*}E_i^{Com}(t), \\ G_{i,j}(t) = -V_1 Z_{i,j}(t) x_{i,j}^C(t), \\ H_{i,j}(t) = -V_1 Z_{i,j}(t) x_{i,j}^G(t), \quad \forall i \in \boldsymbol{\mathcal{I}}, \forall j \in \boldsymbol{\mathcal{J}}, \end{cases} \tag{29}$$

where $X_{i,j}(t)$ only relates to variables $\beta_{i,j,k}(t)$. $G_{i,j}(t)$ relates to $\alpha_{j,k}(TT_0)$, $\beta_{i,j,k}(t)$ and $f_{i,j}^C(t)$. $H_{i,j}(t)$ relates to $\alpha_{j,k}(TT_0)$, $\beta_{i,j,k}(t)$ and $f_{i,j}^G(t)$. $Y_{i,j}(t)$ relates to all variables. Thus when $\alpha_{j,k}(TT_0)$ is decided, problem P2 can be decomposed into four sub-problems

$$\begin{aligned} P3.1: \quad &\min F_{3.1} = \sum_i\sum_j X_{i,j}\left(\beta_{i,j,k}\right) \\ &\text{s.t.} \quad (2)\text{–}(4),\ (10) \text{ and } (20) \end{aligned} \tag{30}$$

$$\begin{aligned} P3.2: \quad &\min\ F_{3.2} \\ &= \sum_i\sum_j Y_{i,j}\left(\beta_{i,j,k}, W_{i,j}^C, W_{i,j}^G, f_{i,j}^C, f_{i,j}^G\right) \\ &\text{s.t.} \quad (2)\text{–}(3) \text{ and } (5)\text{–}(9) \end{aligned} \tag{31}$$

$$\begin{aligned} P3.3: \quad &\min\ F_{3.3} = \sum_i\sum_j G_{i,j}\left(\alpha_{j,k}, \beta_{i,j,k}, f_{i,j}^C\right) \\ &\text{s.t.} \quad (1),\ (8) \text{ and } (9) \end{aligned} \tag{32}$$

$$\begin{aligned} P3.4: \quad &\min\ F_{3.4} = \sum_i\sum_j H_{i,j}\left(\alpha_{j,k}, \beta_{i,j,k}, f_{i,j}^G\right) \\ &\text{s.t.} \quad (1),\ (8) \text{ and } (9) \end{aligned} \tag{33}$$

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

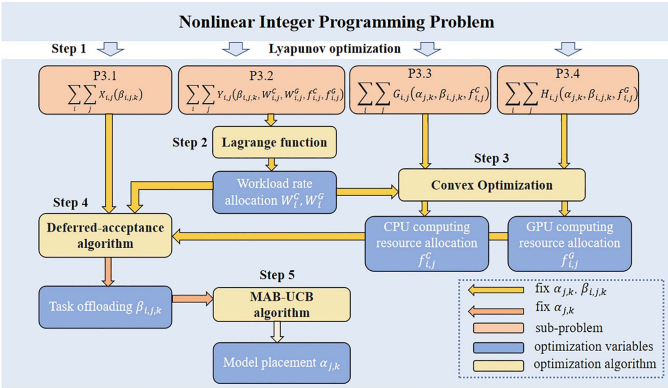


Fig. 4. The overall algorithm process.

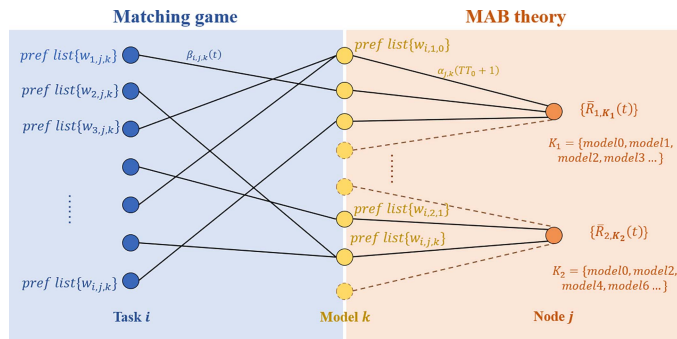


Fig. 5. The overall DA-MAB algorithm.

P3.1 and P3.2 are single time slot problems aimed at maximizing the objective function within each time slot. When $\alpha_{j,k}(TT_0)$ is determined, P3.3 and P3.4 also degrade into single time slot problems. The overall problem will be solved using the proposed DA-MAB algorithm, as illustrated in Fig. 4. When $\alpha_{j,k}(TT_0)$ and $\beta_{i,j,k}(t)$ are fixed, the values of $W^C_{i,j}(t), W^G_{i,j}(t), f^C_{i,j}(t), f^G_{i,j}(t)$ will be determined through the construction of Lagrangian equations and convex optimization. Then when $\alpha_{j,k}(TT_0)$ is fixed, $\beta_{i,j,k}(t)$ will be determined using the DA algorithm, while the decision variables $\alpha_{j,k}(TT_0)$ will be determined using the MAB-UCB algorithm.

B. Optimal Workload Allocation Strategy

First, we get the optimal solution of $W^C_{i,j}(t)$ and $W^G_{i,j}(t)$.

Lemma 1: In order to minimize latency and energy consumption, the relationship between workload and working frequency should be as follows:

$$\frac{W^C_{i,j}(t)}{\gamma^C_i(t) f^C_{i,j}(t)} = \frac{W^G_{i,j}(t)}{\gamma^G_i(t) f^G_{i,j}(t)}, \qquad \forall i \in \mathcal{I}, j \in \mathcal{J}. \tag{34}$$

The proof is provided in Appendix A, available online.

Lemma 2: To minimize latency and energy consumption, the workload should satisfy the following relationship:

$$\frac{\partial Y_{i,j}}{\partial W^C_{i,j}(t)} = \frac{\partial Y_{i,j}}{\partial W^G_{i,j}(t)}, \qquad \forall i \in \mathcal{I}, j \in \mathcal{J}. \tag{35}$$

The proof is provided in Appendix B, available online.

Based on Lemmas 1 and 2, the optimal workload allocation can be obtained as follows:

$$\begin{cases} W^{C*}_{i,j}(t) = \frac{\sum_k \beta_{i,j,k}(t) b_{i,k} LC_i(t) \sqrt{\varepsilon^C_j} \gamma^C_i(t)}{\sqrt{\varepsilon^C_j} \gamma^G_i(t) + \sqrt{\varepsilon^G_j} \gamma^C_i(t)}, \\ W^{G*}_{i,j}(t) = \frac{\sum_k \beta_{i,j,k}(t) b_{i,k} LC_i(t) \sqrt{\varepsilon^C_j} \gamma^G_i(t)}{\sqrt{\varepsilon^C_j} \gamma^G_i(t) + \sqrt{\varepsilon^G_j} \gamma^C_i(t)}. \end{cases} \tag{36}$$

C. The Optimal Working Frequency Allocation Strategy

Next, we will discuss the optimal allocation of working frequency $f^C_{i,j}(t)$ and $f^G_{i,j}(t)$ in P3.2 + P3.3 + P3.4 according to $Z_{i,j}(t)$. Let $W^C_i(t) = \sum_j W^C_{i,j}(t)$, $W^G_i(t) = \sum_j W^G_{i,j}(t)$.

When $Z_{i,j}(t) = 0$, $F_{3.2} + F_{3.3} + F_{3.4} = \frac{V_2 \mu}{T^*} D^{Com}_i(t) + \frac{V_2(1-\mu)\varepsilon^C_j (W^C_i(t))^3}{E^*(\gamma^C_i(t) D^{Com}_i(t))^2} + \frac{V_2(1-\mu)\varepsilon^G_j (W^G_i(t))^3}{E^*(\gamma^G_i(t) D^{Com}_i(t))^2}$. Let $\frac{\partial Y_{i,j}}{\partial D^{Com}_i(t)} = 0$ to get

$$D^{Com*}_i(t) = \sqrt[3]{2A_i(t)}, \tag{37}$$

where $A_i(t) = \frac{(1-\mu)T^*}{E^*\mu}\left(\frac{\varepsilon^C_i (W^C_i(t))^3}{(\gamma^C_i(t))^2} + \frac{\varepsilon^G_i (W^G_i(t))^3}{(\gamma^G_i(t))^2}\right)$.

When $Z_{i,j}(t) \neq 0$, we can get $F_{3.2} + F_{3.3} + F_{3.4} = \sum_i \sum_j Y_{i,j}(f^C_{i,j}(t), f^G_{i,j}(t)) - \sum_i \sum_j G_{i,j}(f^C_{i,j}(t)) - \sum_i \sum_j H_{i,j}(f^G_{i,j}(t))$.

Lemma 3: $F_{3.2} + F_{3.3} + F_{3.4}$ only have one set of $\{f^C_{i,j}(t), f^G_{i,j}(t)\}, \forall i \in \mathcal{I}, j \in \mathcal{J}$, in defining domains $f^C_{i,j}(t) \in (0, +\infty)$, $f^G_{i,j}(t) \in (0, +\infty)$ to take the minimum value of the function.

The proof is provided in Appendix C, available online.

Based on Lemmas 1 and 2, the optimal computing time $D^{Com*}_i(t)$ can be obtained by Cardano formula as follows:

$$D^{Com*}_i(t) = \sqrt[3]{A_i(t) + \sqrt{(A_i(t))^2 + \left(\frac{B_i(t)}{3}\right)^3}} + \sqrt[3]{A_i(t) - \sqrt{(A_i(t))^2 + \left(\frac{B_i(t)}{3}\right)^3}}, \tag{38}$$

where $B_i(t) = \frac{V_1 Z_{i,j}(t) T^*}{V_2 \mu}\left(\sum_k v_{i,k} \alpha_{j,k}(TT_0) \beta_{i,j,k}(t)\right) \cdot \sum_k \beta_{i,j,k}(t) b_{i,k} LC_i(t)$.

According to Lemma 1, it can be concluded that the optional frequency $f^{C*}_{i,j}(t)$ and $f^{G*}_{i,j}(t)$ are both as

$$\begin{cases} f^{C*}_{i,j}(t) = \min\left\{\frac{W^C_{i,j}(t)}{\gamma^C_i(t) D^{Com*}_i(t)}, f^{C,\max}_j\right\} \\ f^{G*}_{i,j}(t) = \min\left\{\frac{W^G_{i,j}(t)}{\gamma^G_i(t) D^{Com*}_i(t)}, f^{G,\max}_j\right\} \end{cases}. \tag{39}$$

D. The Optimal Task Offloading Strategy

For the task offloading problem $\beta_{i,j,k}(t)$ at the short timescale, we utilize the many-to-one matching theory, as illustrated in Fig. 5, to solve the problem in polynomial time. Matching theory provides an efficient and low-complexity approach to solve integer problems and has been widely used in various wireless communication systems.

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

According to the definition of matching theory, matching is an assignment between two sets of players, namely the set of tasks and the set of computing nodes. Considering the structure of problem P2, we consider an extended many-to-one matching model, where each computing node j owns different models and computing resource, can be assigned to multiple tasks, while each task i can only be assigned to one model k on a computing node. The form of the considered many-to-one matching is defined as follows:

Definition 1: The considered many-to-one matching game consists of two sets of participants, namely $\mathcal{I}$ and $\mathcal{J} \times \mathcal{K}$. The result of the many-to-one matching pairs is a function η from all elements of $\mathcal{I}$ to an element of $\mathcal{J} \times \mathcal{K}$ that satisfies:

$$\begin{cases} \beta_{i,j,k}(t) = 1, \ if \ (j,k) = \eta(i), \\ \beta_{i,j,k}(t) = 0, \ otherwise. \end{cases} \tag{40}$$

So P2 can be equivalent to a weighted many-to-one matching problem. For simplicity, the variable t is removed from the following equation. The weight between task i and node (j,k) is given by P2 and the optimal workload allocation and working frequency allocation strategies

$$\begin{aligned} w_{i,j,k} &= V_2 \frac{\mu}{T^*} \left(\frac{d_{i,j} C_i}{R_{i,j}} + D_i^{Com*} \right) \\ &+ V_1 Z_{i,j} \left(b_{i,k} L C_i - \alpha_{j,k} T_{slot} \left[\gamma_i^C f_{i,j}^C + \gamma_i^G f_{i,j}^G \right] \right) \\ &+ V_2 \frac{(1-\mu)}{E^*} \left(\frac{\varepsilon_i^d d_{i,j} C_i}{R_{i,j}} + \varepsilon_j^C W_{i,j}^C \left(f_{i,j}^C \right)^2 + \varepsilon_j^G W_{i,j}^G \left(f_{i,j}^G \right)^2 \right). \end{aligned} \tag{41}$$

Tasks can select computing nodes based on weights arranged in ascending order. Since different tasks require different numbers of CPU and GPU cores, the traditional exchange matching algorithm can only guarantee stability when tasks have the same resource requirements. To address this, we extend the one-sided matching to a two-sided matching. For each task and node, they establish preference lists for each other and use the Gale-Shapley (GS) algorithm, also known as the DA algorithm, to ensure stable matching. The preference list for task i is $\psi_i = \{w_{i,j,k}(t)\}$, $\forall j \in \mathcal{J}, k \in \mathcal{K}$ and (j,k) satisfy (20). The preference list for node j and model k is $\psi_{j,k} = \{w_{i,j,k}(t)\}$, $\forall i \in \mathcal{I}$ and i satisfy (20). The process of DA matching is as follows:

1) Application: Each task applies to its top-ranked computing node in the preference list ψ_i. Each task can only apply to a computing node that has never rejected it before.

2) Selection: Each computing node with capacities $Q_j^{C,\max}$ and $Q_j^{G,\max}$ receives a set of applicants ϕ_j. Due to shared computing capacity among models on one node, the preference lists $\psi_{j,k}$ of all models k deployed in the node are merged into a node preference list ψ_j. Based on the ψ_j, each node accepts a certain number of tasks in ascending order, rejects the remaining tasks by setting their $w_{i,j,k}(t)$ in ϕ_j to $w_{i,j,k}^{\max}(t)$, where $w_{i,j,k}^{\max}(t)$ is a pre-defined penalty threshold larger than all w values. If the total demand of the applicants is less than $Q_j^{C,\max}$ and $Q_j^{G,\max}$, all tasks are accepted.

3) Deferred Acceptance: Rejected tasks continue to apply to computing nodes until they are accepted by a computing node or rejected by all computing nodes in their preference list. Then, each computing node accepts new applicants according to the rules in step 2).

4) Task Delivery: Determine all tasks accepted by computing node j and perform routing for the tasks.

Algorithm 1: Short Timescale Task Offloading and Model Chosen by DA Algorithm.

1: Calculate $w_{i,j,k}(t)$ for all $i \in \mathcal{I}$, $j \in \mathcal{J}$ and $k \in \mathcal{K}$.
2: **repeat**
3: **for** $i \in \mathcal{I}$ **do**
4: Build ψ_i.
5: **end for**
6: **for** $(j,k) \in \mathcal{J} \times \mathcal{K}$ **do**
7: Sort ϕ_j by the weights of ψ_i.
8: **repeat**
9: Set $\eta(i) = (j,k)$ and delete $\eta(i)$ from ϕ_j.
10: **until** $\phi_j = \varnothing$ **or** $Q_j^{C,\max}$ is exceeded **or** $Q_j^{G,\max}$ is exceeded.
11: Set $w_{i,j,k}(t) = w_{i,j,k}^{\max}(t)$ for $\eta(i)$ in ϕ_j.
12: **end for**
13: **until** All tasks are accepted or rejected by all nodes.
Output: Transfer η to the task offloading indicator as (39).

Algorithm 1 provides a detailed description of the steps to solve task offloading problem using DA matching. The complexity of this algorithm depends on the larger value between the number of tasks and the number of computing nodes. Considering that the number of tasks is usually larger than the number of computing nodes in the considered network, the complexity of Algorithm 1 is on the order of $O(I^2)$, increasing quadratically with the number of tasks.

Given the model placement strategy, all optimization problems in the single time slot have been resolved.

E. The Optimal Model Placement Strategy

As shown in Fig. 5, the placement strategy for the computing node models at the long timescale can be reduced to a special case of the MAB problem. In the standard MAB problem, it's assumed there are K arms bandit. The decision-maker has V opportunities to pull the arms, and generates rewards X_v in turn v. The decision-maker does not possess explicit knowledge about the quality of each arm k but relies on past observations to select a potentially good arm that maximizes the cumulative reward $X(t)$.

In our research, after addressing all the problems at the short timescale, our objective can be transformed into solving P4 as

$$\begin{aligned} P4: \min \quad F_4 = &\sum_{t=TT_0}^{(T+1)T_0-1} \sum_i \sum_j \left(G_{i,j} \left(\alpha_{j,k} \left(TT_0 \right) \right) \right. \\ &\left. + H_{i,j} \left(\alpha_{j,k} \left(TT_0 \right) \right) \right) \\ \text{s.t.} \quad &(1) \end{aligned} \tag{42}$$

Due to the lack of knowledge about the arrival of tasks within the time interval $[TT_0, (T+1)T_0)$ at time TT_0, the model selection

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

strategy $\alpha_{j,k}$ can only be adjusted based on historical information within a limited complexity space to maximize long-term rewards.

First, each node j is considered to have $|\mathcal{K}_j|$ arms, where $|\mathcal{K}_j|$ represents the total number of model types deployable on node j. In each round, $\alpha_j^{\max}$ arms need to be pulled for each node j, which means selecting $\alpha_j^{\max}$ different models from the set of model types $\mathcal{K}_j$. (If the domain of $\alpha_{j,k}(t)$ is extended, it means that each arm has a weight $\alpha_{j,k}(t)$, and need to select models that meet (1) instead of selecting $\alpha_j^{\max}$ different models) Second, each pull may result in a change in task selections for the current round, making it a dynamic MAB process. Therefore, we improve the basic Upper Confidence Bounds (UCB) algorithm by updating the confidence bounds of the mean reward at each time slot and selecting the top $\alpha_j^{\max}$ arms with low weights as the current stage's model placement strategy at the beginning of each large time slot.

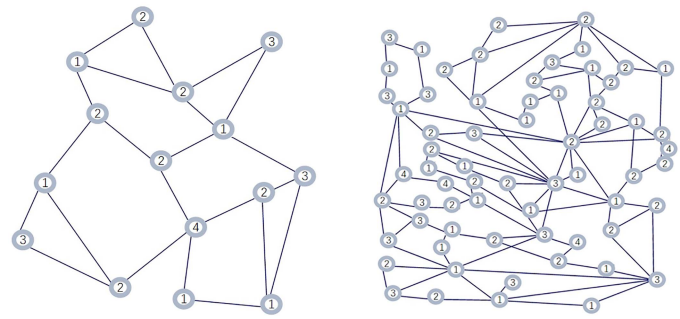
Assuming that at each slot $t \in [TT_0, (T+1)T_0)$, V attempts are made to randomly change the combination of node models. Let $\bar{R}_{j,k}(t) = \frac{\sum_{t=TT_0}^{(T+1)T_0-1} \bar{w}(t)}{v_{j,k}(t)} - w^*(t)$ denote the cumulative regret for arm (j,k) at time t, where $\bar{w}(t)$ represents the average weight value obtained by selecting arm (j,k) in the attempts at time t, $h_{j,k}(t)$ is the cumulative number of attempts to select (j,k) during the time interval $[TT_0, t]$, and $w^*(t)$ is the predefined optimal benchmark utility.

According to the UCB algorithm, the upper boundary of the confidence interval is set to $\sqrt{\frac{2log(V*(t-TT_0))}{h_{j,k}(t)+1}}$, where $V*(t-TT_0)$ represents the total number of selections at time t. Therefore, the cumulative regret for arm (j,k) at time t is given by $\bar{R}_{j,k}(t) - \sqrt{\frac{2log(V*(t-TT_0))}{h_{j,k}(t)+1}}$. It can be observed that when the number of selections $h_{j,k}(t)$ for arm (j,k) is small, the upper boundary of the confidence interval is relatively large, resulting in a lower regret and increasing the probability of selecting that arm for exploration. The probability of exploration is a normalized probability function of cumulative regret. As the number of selections increases, the uncertainty decreases, and the cumulative regret is gradually dominated by $\bar{R}_{j,k}(t)$. At this point, arms with lower cumulative regret will emerge as winners. Based on this, we obtain

$$arm(j) = argmin_k \left[\bar{R}_{j,k}(t) - \sqrt{\frac{2log\left(V*(t-TT_0)\right)}{h_{j,k}(t)+1}} \right]. \tag{43}$$

At $(t+1)T_0$, select the smallest $\alpha_j^{\max}$ regret arms for each node, which is the next long timescale $(t+1)T_0+1$ Model placement strategy. The specific algorithm is as follows:

It can be seen that the complexity of the MAB algorithm is not only related to the number of attempts V, but also to the complexity of the DA algorithm used in step 4 of Algorithm 2. Therefore, in the DA algorithm in step 4, we only perform the first time matching in Algorithm 1 and estimate the total weight to minimize algorithm complexity. So, the complexity of each execution of the MAB algorithm is $O(VT_0)$. At each time slot, the DA algorithm and the MAB algorithm are both executed, and a placement plan is made using the MAB algorithm in fixed interval time slots. The total complexity of executing DA-MAB at time $t \in [TT_0, (T+1)T_0]$ does not exceed $O(I^2T_0 + VT_0)$.



(a) atlanta (b) ta2

Fig. 6. The network topology for simulations.

Algorithm 2: Long Time Scale Model Placement Scheduling by Improved MAB-UCB.

1: **for** $t \in [TT_0, (T+1)T_0]$ **do**
2: **for** $v = 1 : V$ **do**
3: Select arms as $\bar{R}_{j,k}(t) - \sqrt{\frac{2log(V*(t-TT_0))}{h_{j,k}(t)+1}}$.
4: Calculate weight by DA algorithm.
5: Update $\bar{R}_{j,k}(t) = \frac{\sum_{t=TT_0}^{(T+1)T_0-1} \bar{w}(t)}{h_{j,k}(t)+1} - w^*(t)$.
6: **end for**
7: **end for**
8: **if** $t = (T+1)T_0$ **then**
9: **for** $j = 1 : J$ **do**
10: Select $\alpha_j^{\max}$ arms as
$arm(j) = argmin_k \left[\bar{R}_{j,k}(t) - \sqrt{\frac{2log(V*(t-TT_0))}{h_{j,k}(t)+1}} \right]$.
11: **end for**
12: **end if**

V. Simulation Results and Discussions

A. Experimental Settings

In this section, we evaluate the performance of the DA-MAB algorithm. In the basic scenario, we compare the DA-MAB algorithm with other algorithms in the Atlanta network topology with 15 computing nodes, as shown in Fig. 6(a), where the numbers represent the number of models can be deployed for each node. In the experiments, a total of 210 tasks are assigned to 35 users, with task allocation performed every 100 seconds. We assumed that users' tasks followed a periodicity of 600 seconds. For the short time slot analysis, we employed three comparative algorithms: Greedy algorithm, particle swarm optimization (PSO) algorithm, and Proximal Policy Optimization (PPO) algorithm. Additionally, as shown in Fig. 6(b), we use the ta2 network consisting of 65 computing nodes and 560 users to validate the algorithm's performance on a larger topology, referred to as DA-MAB (ta2). The distribution probabilities of tasks and models on this topology are set to be the same as the

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

TABLE II
EXPERIMENTAL PARAMETERS

Data size $C_i(t)$	[5,800] MB
CPU core number requirement of each task $\gamma_i^C(t)$	[0,16] cores
GPU core number requirement of each task $\gamma_i^G(t)$	[0,64] cores
Processing density L	297.62cycles/bit
CPU core number for each computing node $Q_j^{C,max}$	[8,64] cores
GPU core number for each computing node $Q_j^{G,max}$	[32,512] cores
Transmission rate $R_{i,j}(t)$	[50,100] MB/s
Time slot duration T_{slot}	100s
Total time	1200s
Model update interval T_0	600s
Transfer energy coefficient ε_i^d	0.2W
CPU energy coefficient ε_i^C	$5W/(Gps)^3$
GPU energy coefficient ε_i^G	$0.3W/(Gps)^3$

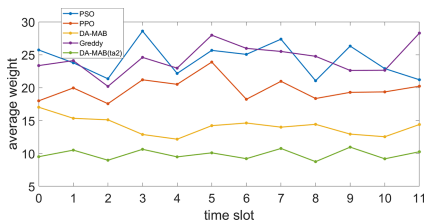


Fig. 7. Overall performance over time.

Atlanta network topology. For the long timescale, commonly used approaches in the communication field involve obtaining prior knowledge through prediction methods and combining it with other techniques for comparison. Therefore, we chose the best-performing PPO algorithm among the aforementioned benchmark algorithms and compared it with our MAB algorithm under the assumption of prior knowledge. The detailed experimental settings are shown in Table II.

B. Overall Performance

First, we compared the performance of the algorithms within each time slot, as shown in Fig. 7. The objective has been equivalent to minimizing the overall weight. It can be observed that our DA-MAB algorithm outperforms the three benchmark algorithms overall, with smaller fluctuations in weight over time. The DA-MAB algorithm with dynamic model placement enabled will be more stable over time, the performance of PSO and PPO and Greedy algorithms is relatively less stable. In the larger ta2 topology, due to stronger node connectivity, the average weight is reduced and more stable compared with the Atlanta network.

C. Analysis of Data Size

Subsequently, we investigated the impact of data size on algorithm performance and optimization results. As shown in Fig. 8, when the data size is increased threefold, DA-MAB, DA-MAB (ta2) and Greedy algorithms exhibit steady growth, while the PSO and PPO algorithms experience significant increases. This is because as the data volume increases, the PSO and PPO algorithms may encounter penalties due to exceeding the node capacity limit, which is inherent to their exploratory choice. The

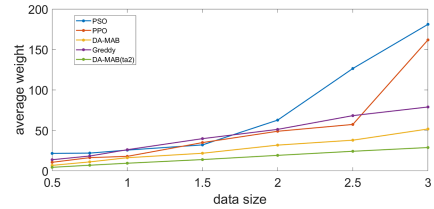


Fig. 8. Single time slot performance with data size varying.

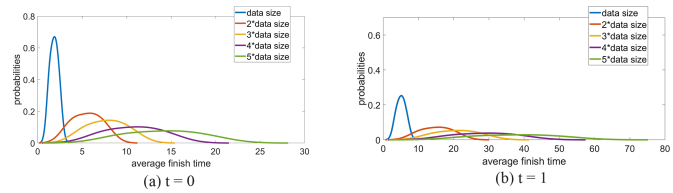


Fig. 9. The probability density distribution of the average task completion.

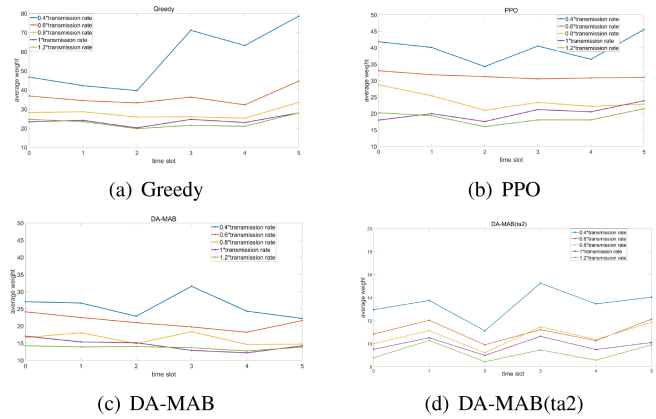


Fig. 10. Performance over time with transmission rate varying.

DA-MAB (ta2) exhibits the slowest growth rate, enabling it can support larger data transmission.

Furthermore, the probability density distribution of the average task completion time for the DA-MAB algorithm under different data sizes is shown in Fig. 9. Fig. 9(a) and (b) represent the probability density distribution at time $t = 0$ and $t = 1$, respectively, where the total data size at $t = 1$ is greater than that at $t = 0$. It can be observed that as the data size increases, the average task completion time increases, including both transmission time and computation time, and shows a more dispersed distribution.

D. Analysis of Transmission Rate

Fig. 10 presents the variation of weight for different algorithms under different transmission rates. The PSO algorithm's performance is weaker than the other three algorithms and is not shown in the figure. Fig. 10(a) illustrates the average weight of rasks for the Greedy algorithm with single time slot optimization across six time slots under different transmission rates. The algorithm performs relatively stable when the transmission rate is high but shows poor performance when the transmission rate is low. This is because at low transmission rates, tasks are more

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

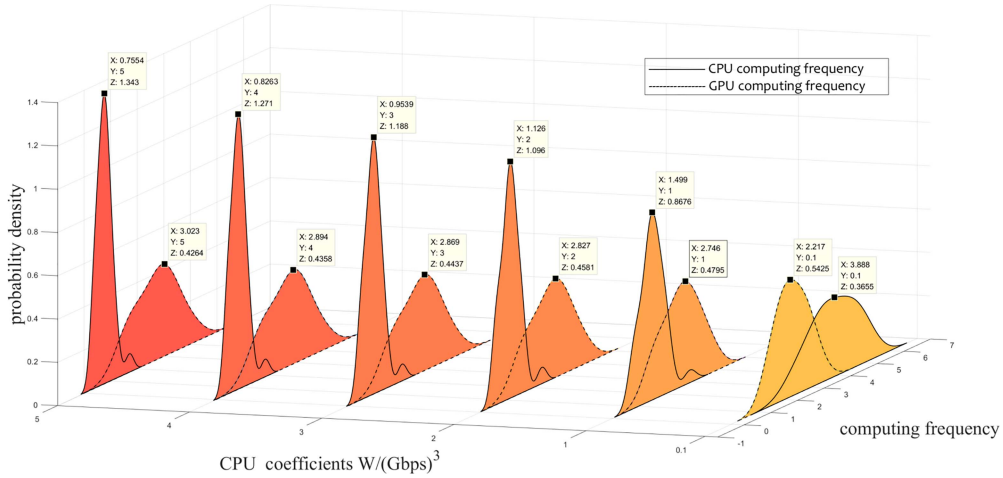
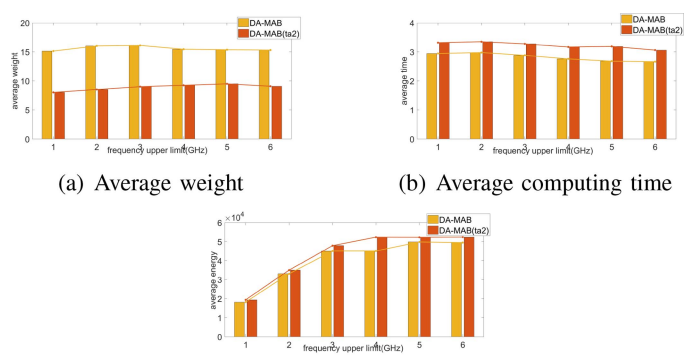


Fig. 11. The probability density distribution of CPU and GPU frequencies with energy coefficient varying.

sensitive to the selection of computing nodes in order to ensure task completion latency. Therefore, the gap between the local optimal solution obtained by the greedy strategy and the global optimal solution becomes larger. Fig. 10(b) shows the average weight of tasks for the PPO algorithm with single time slot optimization. The PPO algorithm exhibits stable performance under various transmission rates, outperforming the Greedy algorithm but slightly inferior to the DA-MAB algorithm. Fig. 10(c) and (d) includes the multi time slot optimization MAB-UCB algorithm, where the MAB algorithm is executed at $t = 3$. It can be observed that after $t = 3$, the performance of DA-MAB is significantly better than the other benchmark algorithms both in small and large network, and the algorithm's performance remains stable under different transmission rates.



(a) Average weight (b) Average computing time

(c) Average energy

Fig. 12. Performance over time with frequency upper limit.

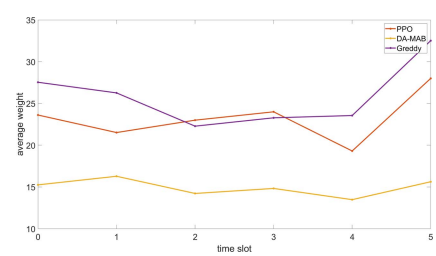


Fig. 13. Performance over time with short communication interval.

E. Analysis of Computing Frequency

Fig. 11 illustrates the impact of the energy coefficient on the optimal selection of CPU and GPU operating frequencies. By changing the energy coefficient of the CPU, the probability density distribution of the CPU and GPU operating frequencies on the same computing node will change. When the CPU energy coefficient decreases, the range of optimal operating frequencies expands, and tasks tend to use higher-frequency CPUs and lower-frequency GPUs. However, the relationship between the two is not linear. When the CPU energy coefficient is lower than that of the GPU, the strategy that optimizes energy consumption and latency comprehensively results in a higher average CPU operating frequency than the average GPU operating frequency. Additionally, the results when changing the GPU energy coefficient are similar to Fig. 11 and are not repeated here.

Fig. 12 illustrates the impact of the frequency upper limit on weight, time and energy. Due to different topological connectivity, the time for comparison only includes computation time. In both network topologies, the weight initially increases and then decreases with the frequency upper limit increasing, while the average computing time gradually decreases and the average energy consumption gradually increases, ultimately stabilizing. It implies that in networks with lower computing power, users take longer to complete tasks but consume less energy. In networks with sufficient computing power, users experience stable average energy consumption while achieving shorter computation times.

F. Analysis of Communication Interval

Then, Figs. 13 and 14 illustrate the impact of shortening the communication interval on the system. The comparison among the three algorithms when T=20s is shown in Fig. 13. Similarly, we enabled the MAB algorithm at $t = 3$, dynamically optimizing the model placement strategy on computing nodes and obtaining better and more stable results in subsequent time slots. Comparing with Fig. 7, it can be observed that when the communication interval is shorter, both the average completion

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

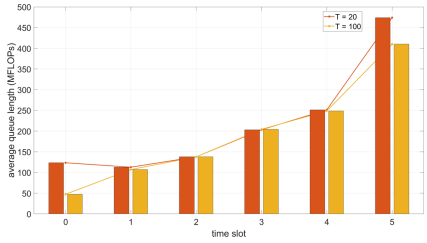
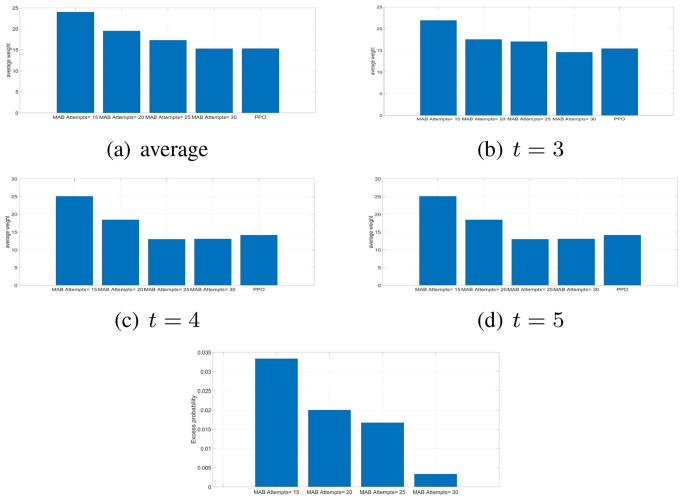


Fig. 14. Average queue length between short and standard communication interval.



(a) average (b) $t = 3$

(c) $t = 4$ (d) $t = 5$

(e) Excess probability

Fig. 15. Performance of MAB algorithm compared with multi time slots PPO.

time and energy consumption of the system are higher than the standard interval. Fig. 14 shows the variation of average queue length for tasks over time under two communication interval conditions. The average queue length for tasks is higher when the communication interval is shorter. Due to the utilization of Lyapunov optimization, which converts the queue stability condition into the optimization objective, the difference in queue length at most time slots is not significant under different communication interval conditions.

G. Analysis of MAB Algorithm

Finally, we discussed the applicability of the MAB algorithm in large-scale model placement strategies. We chose the multi time slots PPO algorithm as the benchmark algorithm, which optimizes model placement based on known information about task workloads at multiple time slots. In Fig. 15(a)–(d), the impact of different numbers of attempts in the MAB algorithm and its comparison with the multi time slots PPO algorithm is shown on the effectiveness of each time slot. Fig. 15(a) shows the average weight of tasks in the three time slots after the model placement strategy is updated. It can be seen that the average weight decreases as the number of attempts increases. When trying 30 times in each time slot, the average weight is already close to that of the PPO algorithm. Fig. 15(b)–(d) compare the optimized weights for the three time slots and their averages. Due to prior global optimization, PPO has more evenly distributed weights in each time slot. However, since the MAB algorithm lacks prior knowledge, its performance in a single time slot may be inferior or superior to the PPO algorithm. Combining with Fig. 15(e), as the number of attempts per round increases, the probability of being penalized for exceeding node capacity gradually decreases for the MAB algorithm, and there are almost no cases of exceeding node capacity when trying 30 times per round. From this, we can conclude that the MAB algorithm, as the number of attempts increases, reduces weight by selecting stable strategies based on accumulated experience and may achieve better strategies than the prior PPO algorithm within a single time slot.

In addition, it should be noted that although the MAB algorithm provides a distributed model placement strategy, the employed DA algorithm is implemented in a centralized manager to achieve low-complexity strategy updates. We are going to improve task offloading strategy to achieve a fully distributed, low-complexity, and efficient solution in the future work.

VI. Conclusion

In this paper, we construct an AIaaS-based computing power network resource allocation framework, which enables dynamic model placement, computing power allocation and task offloading strategies. The long-term problem of minimizing overall network latency and energy consumption is transformed into short-term problems by Lyapunov optimization. We employ an improved deferred-acceptance algorithm, convex optimization, and a multi-armed bandits algorithm to achieve low-complexity online optimization. Finally, extensive experiments are conducted to evaluate the performance of the proposed algorithms compared to existing approaches. In the future work, we consider developing the testbed for the proposed framework and validating the algorithm in the real world.

References

[1] X. Tang et al., "Computing power network: The architecture of convergence of computing and networking towards 6G requirement," *China Commun.*, vol. 18, no. 2, pp. 175–185, 2021.

[2] Recommendation I, "Framework and overall objectives of the future development of IMT for 2030 and beyond," *Int. Telecommun. Union (ITU) Recommend. (ITU-R)*, 2023.

[3] E. Peltonen et al., "6G white paper on edge intelligence," 2020, *arXiv: 2004.14850*.

[4] D. Xu et al., "Edge intelligence: Empowering intelligence to the edge of network," in *Proc. IEEE*, vol. 109, no. 11, pp. 1778–1837, Nov. 2021.

[5] W. Zhang, S. Zeadally, W. Li, H. Zhang, J. Hou, and V. C. M. Leung, "Edge AI as a service: Configurable model deployment and delay-energy optimization with result quality constraints," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1954–1969, Second Quarter, 2023.

[6] M. Latva-Aho and K. Leppänen, "Key drivers and research challenges for 6G ubiquitous wireless intelligence," Univ. Oulu, While Paper, Oulu, Finland, 2019. [Online]. Available: http://urn.fi/urn:isbn:9789526223544

[7] H. Geng, D. Zeng, and Y. Li, "Performance efficient layer-aware DNN inference task scheduling in GPU cluster," in *Proc. IEEE Glob. Commun. Conf.*, 2022, pp. 2242–2247.

[8] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Jan. 2019.

[9] B. Costa et al., "Orchestration in fog computing: A comprehensive survey," *ACM Comput. Surv.*, vol. 55, no. 2, pp. 1–34, 2022.

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.

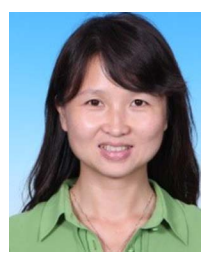
[10] H. Kokkonen, S. Pirttikangas, and L. Lovén, "EISim: A platform for simulating intelligent edge orchestration solutions," 2023, *arXiv:2311.01224*.
[11] H. Kokkonen et al., "Autonomy and intelligence in the computing continuum: Challenges, enablers, and future directions for orchestration," 2022, *arXiv:2205.01423*.
[12] J. Li et al., "AIQoSer: Building the efficient inference-QoS for AI services," in *Proc. IEEE/ACM 30th Int. Symp. Qual. Service*, 2022, pp. 1–10.
[13] B. Da and M. Carugi, "Representative use cases and key network requirements for network 2030," Tech. Rep. ITU-T FG-NET2030, 2020.
[14] E. Peltonen et al., "The many faces of edge intelligence," *IEEE Access*, vol. 10, pp. 104769–104782, 2022.
[15] M. Chen and Y. Hao, "Task offloading for mobile edge computing in software defined ultra-dense network," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 3, pp. 587–597, Mar. 2018.
[16] S. Deng, H. Zhao, W. Fang, J. Yin, S. Dustdar, and A. Y. Zomaya, "Edge intelligence: The confluence of edge computing and artificial intelligence," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7457–7469, Aug. 2020.
[17] A. Bhuiyan, D. Liu, A. Khan, A. Saifullah, N. Guan, and Z. Guo, "Energy-efficient parallel real-time scheduling on clustered multi-core," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 9, pp. 2097–2111, Sep. 2020.
[18] Q. Zeng, Y. Du, K. Huang, and K. K. Leung, "Energy-efficient resource management for federated edge learning with CPU-GPU heterogeneous computing," *IEEE Trans. Wireless Commun.*, vol. 20, no. 12, pp. 7947–7962, Dec. 2021.
[19] J. Feng, W. Zhang, Q. Pei, J. Wu, and X. Lin, "Heterogeneous computation and resource allocation for wireless powered federated edge learning systems," *IEEE Trans. Commun.*, vol. 70, no. 5, pp. 3220–3233, May 2022.
[20] K. Cao, Y. Cui, L. Li, J. Zhou, and S. Hu, "CPU-GPU cooperative QoS optimization of personalized digital healthcare using machine learning and swarm intelligence," *IEEE/ACM Trans. Comput. Biol. Bioinf.*, vol. 21, no. 4, pp. 521–533, Jul./Aug. 2024.
[21] Z. Zhang, Q. Li, L. Lu, D. Guo, and Y. Zhang, "Joint optimization of the partition and scheduling of DNN tasks in computing and network convergence," *IEEE Netw. Lett.*, vol. 5, no. 2, pp. 130–134, Jun. 2023.
[22] S. Tuli, G. Casale, and N. R. Jennings, "SplitPlace: AI augmented splitting and placement of large-scale neural networks in mobile edge environments," *IEEE Trans. Mobile Comput.*, vol. 22, no. 9, pp. 5539–5554, Sep. 2023.
[23] T. Lähderanta et al., "Edge computing server placement with capacitated location allocation," *J. Parallel Distrib. Comput.*, vol. 153, pp. 130–149, 2021.
[24] X. Wei, A. B. M. M. Rahman, D. Cheng, and Y. Wang, "Joint optimization across timescales: Resource placement and task dispatching in edge clouds," *IEEE Trans. Cloud Comput.*, vol. 11, no. 1, pp. 730–744, First Quarter, 2023.
[25] Y. He et al., "Two-timescale resource allocation for automated networks in IIoT," *IEEE Trans. Wireless Commun.*, vol. 21, no. 10, pp. 7881–7896, Oct. 2022.
[26] Y. Sun, S. Chen, Z. Wang, and S. Mao, "A joint learning and game-theoretic approach to multi-dimensional resource management in fog radio access networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 2, pp. 2550–2563, Feb. 2023.
[27] W. Fan et al., "DNN deployment, task offloading, and resource allocation for joint task inference in IIoT," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1634–1646, Feb. 2023.
[28] M. Volk et al., "Facing big data system architecture deployments: Towards an automated approach using container technologies for rapid prototyping," in *Proc. 55th Hawaii Int. Conf. Syst. Sci.*, 2022, pp. 6125–6134.
[29] M. J. Neely, "Stochastic network optimization with application to communication and queueing systems," *Synth. Lectures Commun. Netw.*, vol. 3, no. 1, pp. 1–211, 2010.

Yuhao Chai received the BE degree from the Beijing University of Posts and Telecommunications, Beijing, China, where he is currently working toward the PhD degree with the School of Electronic Engineering. His research interests include 6G network, game theory, machine learning and network intelligence.

Kaice Gao received the BE degree from the Beijing University of Posts and Telecommunications, Beijing, China, where he is currently working toward the MS degree with the School of Electronic Engineering. His research interests include computing power network, distributed training and reinforcement learning.

Guohan Zhang received the PhD degree in management mathematics from the University of Birmingham, United Kingdom, in 2017. He is currently an assistant professor with the Department of Mathematics, the School of Science, BUPT, Beijing China and Key Laboratory of Mathematics and Information Networks (Beijing University of Posts and Telecommunications), Ministry of Education, China. His research interests include mathematical optimization, image processing and machine learning.

Lu Lu (Member, IEEE) received the master's degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2005. She is the deputy director with the Department of Basic Network Technology, China Mobile Research Institute, Beijing, the leader of Core Network Group of CCSA TC5, and the vice-chairman of ITU-T SG13. Her main research direction covers 5G/6G architecture and computing power network.



Qin Li received the master's degree from Chongqing University, Chongqing, China, in 2002. She is a senior researcher with China Mobile Research Institute, Beijing, China, and has engaged in technical and application research on core network and content network for more than 20 years. Her research interests cover 5G/6G architecture, network intelligence and digital twin network.

Yong Zhang received the PhD degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2007. He is a professor with the School of Electronic Engineering, BUPT. He is currently the director of Fab.X Artificial Intelligence Research Center, BUPT. He is the deputy head of the mobile internet service and platform working group, China communications standards association. He has authored or coauthored more than 80 papers and holds 30 granted China patents. His research interests include artificial intelligence, wireless communication, and Internet of Things.

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on March 26,2026 at 09:09:06 UTC from IEEE Xplore. Restrictions apply.